

Learning Multi-Agent Steering Behavior from Images

1. Summary

This project asks whether an algorithm, rather than a modeler, can discover how agents should move — and whether it can do so starting from a picture. A scene image is parsed by a convolutional network into a structured description, and a reinforcement-learning policy drives every agent in that scene from a local, ego-centric observation. The output is the path each agent walks to its goal.

The scene generator is built and validated (Section 4). The remaining work is four experiment families (Section 5).

2. Scope

Four scope decisions were settled:

Decision	Value
Agents per scene	Multiple. Agents avoid obstacles, walls <i>and each other</i>
Sensing geometry	From the research statement: 220° field of view, range 22 in a 100×100 world
First scenario	Obstacle avoidance , and not restricted to hallway layouts
Metrics	Both families — success / collision / timeout rates, and the collision-violation, finish-time and remaining-agent counters used in the existing framework

3. Approach

A two-stage pipeline. Both stages train from the simulator alone: the renderer emits pixel-perfect labels, and the policy learns from reward. No hand-labeled data and no hand-written steering rules enter the system.

Stage	Component	Input → Output
1	Scene image	512×512 PNG
2	U-Net pixel classifier	image → 6-class label map
3	Geometry extraction	label map → agent poses, obstacles, walls, regions
4	Scene instantiation	structured description → simulator state
5	Ego-centric observation	simulator state → 51 numbers per agent per step
6	Shared policy (PPO)	observation → (turn, speed)
7	Synchronous simulation	actions → trajectories
8	Path rendering	trajectories → paths drawn over the input

The observation is what carries generality, and what makes the comparison possible. Each agent sees 16 field-of-view sectors reported separately for three entity categories — obstacles, walls, other agents — plus goal distance, goal bearing and its own speed. Distances are clipped and normalized, so the vector never encodes absolute position.

All agents share one policy, matching the homogeneous agent setting of the prior work. Agents step synchronously: every action is computed from the same world state, then applied together.

4. Infrastructure

The scene generator and simulator are complete and validated.

Scene generation. Seven layout families — open, corridor, room, doorway, two-doorway, cul-de-sac, and a plus-shaped crossing. Each scene is written as an RGB image, a label map whose pixel values are class indices, and a JSON ground truth with exact agent poses, obstacle geometry, wall segments and regions.

Validation. Every scene is checked in configuration space before it is written: obstacles are inflated by the agent radius via a Euclidean distance transform, so "a disk of radius R fits here" is exact rather than approximated on a grid. The tightest passage from start to goal is measured and recorded with the scene, giving every sample a difficulty label. Obstacles are placed one at a time with the check re-run after each, so a scene can never be sealed shut.

5. Experiment plan

We start from a null baseline first. A greedy controller that just turns toward the goal at full speed, run on 20 scenes per layout with 4 agents and 3 obstacles. Its success rate falls from 56% on open scenes to 4% on the crossing, so the suite spans a wide range of difficulty and the trivial strategy solves almost none of it.

Layout family	Greedy success rate
open	56%
room	34%
two-doorway	12%
doorway	6%
corridor	5%
cul-de-sac	5%
crossing	4%

Six experiments follow, numbered in the order they run. Each answers one question. E2 produces the baseline policy that every later experiment builds on, and E3 fixes the reward before the policy is studied further.

	Experiment	Question
E1	Perception robustness	How reliable is the image parser, and what do its errors cost?
E2	Baseline policy	Can one shared policy learn to reach the goal on all seven layouts?
E3	Reward sensitivity	How much does the choice of reward change the learned behavior?
E4	Layout transfer	Does a policy work on layouts it never trained on?
E5	Crowd size	Does the policy still work with 2, 4, 8 and 16 agents?
E6	Comparison with discovered models	How do learned policies compare with the genetically discovered ones?

E1 — Perception robustness

Train the pixel classifier at 128, 256 and 512 resolution and with varying augmentation (colour, line width, background, noise, blur). Then run the same policy on parsed geometry and on exact ground truth. The gap between the two is the cost of perception error, and shows whether effort should go into perception or into the policy.

Multi-agent scenes are the hard case: close agents merge into one outline. Since the generator controls spawn spacing, detection error is plotted against agent distance, which shows exactly where the parser breaks.

E2 — Baseline policy

Build the multi-agent RL environment and train one shared policy with the default reward on all seven layout families at 4 agents. This is the first result of the project: it shows whether the observation and reward are enough for a policy to learn at all, and how far it gets above the greedy baseline on each layout. The trained policy, its training curves and its per-layout scores are the reference point for everything that follows.

E3 — Reward sensitivity

This experiment measures how much that choice matters, and settles the reward before the policy is studied further. Starting from the E2 setup, vary the progress term, collision penalty, step cost, agent-collision penalty and a clearance bonus, and record how behavior changes: distance kept from obstacles, willingness to pass between two obstacles, path length against safety. It is the learned-policy version of asking how sensitive genetic search is to its fitness function. The reward that comes out of this experiment is used in E4 to E6.

E4 — Layout transfer

E2 trains on all seven layouts. This experiment tests on all seven, including families never seen in training. Report results per layout and split by bottleneck width, so a narrow-passage failure is not mistaken for a wrongly learned behavior. The E2 policy is the upper reference: it shows what the same policy achieves when it has seen every layout.

E5 — Crowd size

Train and evaluate with 2, 4, 8 and 16 agents. Report collision violations and finish time next to success rate, and test whether a policy trained at one crowd size still works at another. The 4-agent condition is E2 and is not retrained.

E6 — Comparison with discovered models

This is the comparison that motivates the project. Score the learned policies on the same scenarios, metrics and protocol as the existing framework's obstacle-avoidance models. Then compare the paths, not just the numbers: do the two methods find similar solutions, or different ones?

A second step distills the policy into a small decision tree over the same observation, reporting the rules and the performance lost. If the tree reduces to "steer toward the widest open sector", that is itself a result: two very different search methods reaching the same behavior.

6. Computing resources requested

6.1 GPU for perception training

The classifier is a compact U-Net. A single run of 60 times on 5,000 photos at a resolution of 256×256 requires around 14 hours utilizing 8 CPU threads. A mid-tier GPU operates at a speed 20 to 40 times greater, resulting in a runtime of 20 to 40 minutes per execution. The approximately 45 runs of E1 require around 23 GPU-hours or more than 500 CPU hours.

6.2 CPU for policy training

The policy operates within a limited network, resulting in training time being mostly influenced by environment simulation on the CPU. The environment operates at 14,000 agent-steps per second for each core; however, when accounting for policy inference and updates, a more realistic rate is between 4,000 and 6,000 agent-steps per second. Expenditure increases approximately in a linear manner with the

number of agents, therefore the budget is determined by the extent of experience required by the policy, rather than the density of the scenes. None of these runs depend on each other, so they can all run at once. On 32 cores a whole experiment stage finishes in under two hours.

6.3 Data generation and evaluation

Scene generation runs at 20 scenes/s on one core, under 5 core-hours totally. Evaluation (100 episodes $\times$ 7 layouts $\times$ 5 seeds per condition) takes about 10 minutes per condition, around 20 core-hours in total.

6.4 Storage

One scene is 15.4 KB with image, label map and ground truth, so 100,000 scenes fit in 2 GB. Checkpoints and logs add little. A total of 50 GB is required at most.

6.5 Totals

Resource	Requested	Basis
GPU-hours	40	45 perception runs $\times$ 20–40 min (6.1)
GPU class	A10 / RTX 4070	1.09M-parameter U-Net at 256 $\times$ 256
CPU core-hours	350	$\sim$ 100 independent 35-minute single-core jobs; one experiment stage $\approx$ 5 hours on 12 cores
CPU class	16+ threads	Large amount of short independent jobs
Storage	50 GB	15.4 KB per scene, plus checkpoints and logs (6.4)

7. Software environment

Python 3.11 or newer. The simulator needs only NumPy, OpenCV and Pillow, and runs on CPU. PyTorch is used for the classifier and Stable-Baselines3 for PPO. No licensed software, container registry or external data transfer is needed; every dataset is generated locally.

8. Milestones

Weeks	Work	Output
1	Scene generator, validation, tests, documentation	partially done
2–3	Perception pipeline; E1 resolution and augmentation experiments	perception indicators; error versus spacing graph
4–5	E2 baseline policy on all seven layouts, per-layout scores	training curves; default policy
6–7	E3 reward	better general policy
8–9	E4 layout transfer	model generalization ability

Weeks	Work	Output
10	E5 crowd size	results at 2 / 4 / 8 / 16 agents
11–12	E6 comparison	aligned comparison
13	Distil the policy into readable rules	rule set and its cost
14	Write-up	report and figures

9. Deliverables

- A validated multi-agent scene generator and simulator, with documentation and tests.
- A perception module that reconstructs scene structure from photos automatically, with detection inaccuracies noted.
- One shared steering policy learned from reward, evaluated on seven or more layout families.
- A measured account of how much the reward’s design shapes the learned behavior.
- A comparison between genetic algorithm models and learned policies.

References

- Keller, N., and X. Hu. 2019. "Towards Data-Driven Simulation Modeling for Mobile Agent-Based Systems." *ACM Transactions on Modeling and Computer Simulation* 29(1):1–26.
- Le, H., and X. Hu. 2020. "Extended Model Space Specification for Mobile Agent-Based Systems to Support Automated Discovery of Simulation Models." In *Proceedings of the 2020 Winter Simulation Conference*, 2233–2244. IEEE.
- Le, H., and X. Hu. 2022. "Automated Model Discovery for Steering Behavior Simulation." In *Proceedings of the 2022 Annual Modeling and Simulation Conference (ANNSIM '22)*, 54–65. SCS.
- Raffin, A., A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. 2021. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." *Journal of Machine Learning Research* 22(268):1–8.
- Reynolds, C. W. 1999. "Steering Behaviors for Autonomous Characters." In *Proceedings of the Game Developers Conference 1999*, 763–782.
- Ronneberger, O., P. Fischer, and T. Brox. 2015. "U-Net: Convolutional Networks for Biomedical Image Segmentation." In *MICCAI 2015*, 234–241. Springer.
- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. 2017. "Proximal Policy Optimization Algorithms." arXiv:1707.06347.
- Tai, L., G. Paolo, and M. Liu. 2017. "Virtual-to-Real Deep Reinforcement Learning: Continuous Control of Mobile Robots for Mapless Navigation." In *IEEE/RSJ IROS 2017*, 31–36.

Tobin, J., R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. 2017. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World." In *IEEE/RSJ IROS 2017*, 23–30.